

Name: _____

Class: _____

**JURONG PIONEER JUNIOR COLLEGE****JC2 Preliminary Examination 2026****COMPUTING
Higher 2****9569/02****21 August 2026**

Paper 2 (Practical)

3 hours

Additional materials:

Cover Page
Electronic version of TASK1.txt
Electronic version of STAFF.txt
Electronic version of HOUSE.txt
Electronic version of aquila.jpg
Electronic version of aries.jpg
Electronic version of pavo.jpg
Electronic version of taurus.jpg
Electronic version of ursula.jpg
Electronic version of volans.jpg
Insert Quick Reference Guide

READ THESE INSTRUCTIONS FIRSTAnswer **all** the questions.

All tasks must be done in the computer laboratory. You are not allowed to bring in or take out any pieces of work or materials on paper or electronic media or in any other form.

Approved calculators are allowed.

Save each task as it is completed.

The use of built-in functions, where appropriate, is allowed for this paper unless stated otherwise.

Note that up to **6** marks out of 100 will be awarded for the use of common coding standards for programming style.

The number of marks is given in brackets [] at the end of each task.

The total number of marks for this paper is 100.

This document consists of **13** printed pages and **3** blank pages.**[Turn over**

Instructions to candidates:

Your program code and output for each of Task 1 to Task 4.3 should be downloaded in one `.ipynb` file per task. For example, your program code and output for Task 1 should be downloaded as `TASK1_<your name>_<your class>.ipynb`.

1 Name your Jupyter Notebook as:

`TASK1_<your name>_<your class>.ipynb`.

JPMart, a local supermarket chain, is developing an inventory management program to help store managers organise and search for products efficiently. Each product has the following attributes:

- `productID: INTEGER`
- `name: STRING`
- `price: FLOAT`
- `stockQuantity: INTEGER`

The system must allow the manager to view, sort, and search the inventory data based on different needs.

Task 1.1

Write a function `task1_1(filename)` that:

- takes a string `filename` which represents the name of a text file
- reads in the contents of the text file
- stores each line in the text file in a tuple, with each attribute stored using its data type shown above
- returns the list of tuples.

[3]

Call your function `task1_1` with the file `TASK1.txt`, printing the returned list, using the following statements:

```
result = task1_1("TASK1.txt")
print(result)
```

[1]

Task 1.2

One method of sorting is the bubble sort.

Write a function `task1_2(list_of_records)` that:

- takes a list of tuples
- implements a bubble sort algorithm to arrange the products in ascending order of their **price**
- returns the sorted list of tuples.

[4]

Call your function `task1_2` with the contents of the file `TASK1.txt`, printing the returned list, using the following statement:

```
print(task1_2(task1_1("TASK1.txt")))
```

[1]

Task 1.3

Another sorting method is known as quick sort.

Write a function `task1_3(list_of_records)` that:

- takes a list of tuples
- implements a quick sort algorithm to arrange the products in ascending order of their **product ID**
- returns the sorted list of tuples.

[4]

Call your function `task1_3` with the contents of the file `TASK1.txt`, printing the returned list, using the following statement:

```
print(task1_3(task1_1("TASK1.txt")))
```

[1]

Task 1.4

Write a program code to declare a recursive function to:

- take in a **product ID** and the sorted list of tuples from **Task 1.3** as two of its parameters
- implement recursive binary search
- return the name, price and stock quantity of the product if found
- return "Not Found" if the product ID is not in the list.

Write a `main()` function to:

- get user to input the product ID
- call the binary search function with the integer value of the product ID
- output the result

[4]

Call the `main()` function **two** times to test with the following inputs:

Test 1: largest product ID in the list

Test 2: one product ID that is **not** in the list.

Show the output of each test.

[1]

Save your Jupyter Notebook for Task 1.

[Turn over

2 Name your Jupyter Notebook as:

TASK2_<your name>_<your class>.ipynb.

You are asked to implement a 2-dimensional game called **Treasure Game** using Object-Oriented Programming.

g.	col 0	col 4			
row 0	P	.	.	.	.
	.	.	T	.	.
	.	.	.	B	T
	.	.	.	T	.
row 4	.	.	.	.	.

The diagram above shows a 5 by 5 grid, which represents the game map.

The player starts at the grid position row 0, column 0, which is at the top left-hand corner of the map, as shown in the diagram above.

The map contains:

- Normal Tile (.)
- Treasure (T)
- Bomb (B)
- Player (P)

To complete the challenge, the player must **collect all three treasures** (by moving to the coordinates of the treasure tiles).

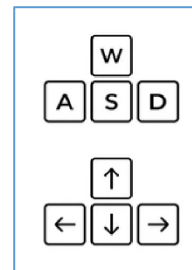
If the player touches a bomb, it will lose some lives.

The controls are:

W
A S D

where,

W - Move Up
A - Move Left
S - Move Down
D - Move Right



The base class `Tile` contains:

- the attribute `symbol`, a character to be displayed to represent the tile
- a constructor that takes `symbol` as parameter to set the attribute
- the method `get_symbol()` that returns the symbol.

The `Treasure` class (a subclass of `Tile`) contains:

- the attribute `treasure` that is a string description of the treasure
- a constructor that takes `symbol` and `treasure` as parameters to set the attributes
- the method `get_treasure()` that returns the `treasure`.

The `Bomb` class (a subclass of `Tile`) contains:

- the attribute `damage` that is an integer to represent the number of lives the player will lose if player touches the tile
- a constructor that takes `symbol` and `damage` as parameters to set the attributes
- the method `get_damage()` that returns the value in `damage`

Task 2.1

Write the class definitions for the `Tile`, `Treasure` and `Bomb` classes.

[6]

Task 2.2

Write the `display_grid(grid)` function that takes in `grid`, that is a 2D array of `Tile` objects, and display the symbols on the `Tile` objects.

[3]

Task 2.3

Write the `main()` function for the game as follows:

1. Create `grid`, a 2D array that has 5 rows and 5 columns, that contains `Tile` objects with the `symbol` `'.'`
2. Create `Treasure`, `Bomb` and `player` `Tile` objects, and store them in the respective positions in the 2D array `grid` according to the information as follows:

Object type	row, col	symbol	treasure	damage
Treasure	1, 2	'T'	"Silver key"	
Treasure	2, 4	'T'	"Gold coin"	
Treasure	3, 3	'T'	"Jade pendant"	
Bomb	2, 3	'B'		1
Tile	0, 0	'P'		

3. Call the `display_grid` function from Task 2.2 to display the grid.
4. Create suitable variables and initialise them accordingly so that the player starts at the **top-left corner** of the grid, has an **empty inventory list**, and has **one life**.
5. Prompt the user to enter a sequence of moves (W/A/S/D) and store as a string.
6. For each move in the sequence of moves,
 - a. Use the representation below to compute the new location:
 - 'W' moves up
 - 'A' moves left
 - 'S' moves down
 - 'D' moves right
 - b. If the new location lies within the boundaries of the 2D grid,
 - i. update the player's row and column variables to the new location, and access the symbol stored in the corresponding cell.
 - ii. If the symbol is **B**, reduce the player lives according to the damage value of the `Bomb` object.
 - iii. Else if the symbol is **T**, add the treasure of the `Treasure` object to the player inventory list, provided there is no duplicate treasure in the inventory.

Ignore any move that goes outside the grid.

[Turn over]

7. At the end of the game, display an appropriate message based on the outcome.
- If player has no lives remaining, display: `Game Over, no more lives.`
 - Otherwise, if all three treasures have been collected, display:
`Congratulations! You won the game! Display the number of treasures collected and the names of the collected treasures`
 - Otherwise, display: `Sorry, you did not win the game. Display the number of treasures collected and the names of the collected treasures.` [14]

Sample run 1 (Won the game):

```
P . . . .
. . T . .
. . . B T
. . . T .
. . . . .

Enter sequence of moves (W/A/S/D): SDDDDSSA

Congratulations! You won the game!
Treasures collected 3/3: Silver key, Gold coin, Jade pendant
```

Sequence of moves for sample run 1 :

P	.	.	.	.
.	.	T	.	.
.	.	.	B	T
.	.	.	T	.
.	.	.	.	.

SDDDDSSA

Sample run 2 (Did not win):

```
P . . . .
. . T . .
. . . B T
. . . T .
. . . . .

Enter sequence of moves (W/A/S/D): WASDDDDSSSA
Sorry, you did not win the game.
Treasures collected 2/3: Silver key, Gold coin
```

Sequence of moves for sample run 2 :

P	.	.	.	.
.	.	T	.	.
.	.	.	B	T
.	.	.	T	.
.	.	.	.	.

WASDDDDSSSA
 ↑
 Out of boundary moves are ignored.

Sample run 3 (Game Over, no more lives):

```

P . . . .
. . T . .
. . . B T
. . . T .
. . . . .

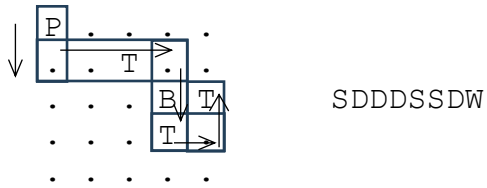
```

```

Enter sequence of moves (W/A/S/D): SDDDSSDW
Game Over, no more lives.

```

Sequence of moves for sample run 3:



Call the `main()` program **three** times, and test with the above three set of sample inputs.

[2]

Show the **three** sets of output.

Save your Jupyter Notebook for Task 2.

[Turn over

3 Name your Jupyter Notebook as:

TASK3_<your name>_<your class>.ipynb.

Jurong Pioneer Bird Park (JPBP) deploys hundreds of environmental sensors to monitor the birds' movements and their habitat conditions.

Every environmental sensor is assigned to a unique five-digit string `SensorID` that ranges from '00000' to '99999'. For the `SensorID` of the sensors deployed in JPBP to be accessed efficiently, each `SensorID` is stored in a node of a Binary Search Tree (BST).

Each node in the BST is implemented using a dictionary as shown in the following example:

```
{"Left": None, "SensorID": "58342", "Right": None}
```

The variable `Root` stores the root node of the BST.

Task 3.1

Write a Python function `createSensorNode()` that:

1. Generates a unique `sensorID` by randomising an integer from 0 to 99999, inclusive.
2. Converts the randomly generated integer from step 1 into a five-digit string by adding leading zeros where necessary.
3. Returns a BST node represented by a dictionary containing the five-digit string obtained in step 2.

The returned node should have the following format:

```
{"Left": None, "SensorID": sensorID, "Right": None}
```

For example, if the generated integer is 123, the `sensorID` would be "00123" and the function should return:

```
{"Left": None, "SensorID": "00123", "Right": None}
```

[5]

Task 3.2(a)

Write a function `insert (Root, newSensorNode)` that inserts `newSensorNode` into the BST referenced by its root node `Root`, and return the root node.

The function should traverse the BST starting from its `Root` and insert `newSensorNode` at the appropriate position such that the BST is in ascending alphanumeric order of the `SensorID` of the node, and return the root node of the updated BST.

If BST is initially empty, `newSensorNode` should become the root node and returned.

Assume that the `SensorID` of every `newSensorNode` is unique (i.e. it does not already exist in the BST).

[4]

Task 3.2(b)

In practice, the `SensorID` of a `newSensorNode` may already exist in the BST (i.e. a duplicate may occur).

Write a function `insertHandleDuplicate (Root, newSensorNode)` that is similar to `insert` function in Task 3.2(a), but add in the check whether the `SensorID` of `newSensorNode` is equal to the `SensorID` of the node being compared against.

If the `SensorID` is the same, a new sensor node should be generated. The entire insert process should then restart from `Root` using this newly generated node, since the new `SensorID` may belong anywhere in the BST.

[2]

Task 3.3

Write a procedure `inOrder (Root)` that performs an in-order traversal of the BST referenced by `Root` and displays the `SensorID` of each environmental sensor in ascending alphanumeric order.

[4]

[Turn over]

Task 3.4

JPBP has recently purchased 10 new environmental sensors.

In your main program:

1. Initialise an empty BST by setting its root node `TRoot`, to `None`.
2. For each of the 10 newly procured sensors,
 - call the function `createSensorNode()` to assign a `SensorID` to it and create a sensor node `newSensorNode`
 - insert the new node into the BST by calling:


```
TRoot = insert(TRoot, newSensorNode)
```

Test your program and show the output.

[4]

Task 3.5

A survey done by the programmers showed that JPBP staff frequently entered `SensorIDs` erroneously when they wanted to search for a specific environmental sensor by `SensorID` in the program.

Write a Python function `isValidSensorID(sensorID)` that returns `True` only if the string parameter `sensorID`:

- contains exactly five characters; and
- consists only of digits from 0 to 9.

The function should return `False` otherwise.

[2]

Task 3.6

Write the program for **three** test cases to test the function `isValidSensorID`.

For each test case, show:

- the test data (in the input argument)
- the test type, normal, boundary or erroneous, and reasoning (in comments)
- the expected output (in comments)

Test your program and show the output.

[3]

Save your Jupyter Notebook for Task 3.

4 Name your Jupyter Notebook as:

TASK4_<your name>_<your class>.ipynb.

JP runs six Houses and wishes to implement a web-based house allocation system to streamline staff allocation and maintain accurate records.

Staff will submit their name and department in a web form. Upon submission, the system will allocate a House, display the House's image, and update the database. The system should support the storage of staff details, House information, and allocation.

The database will have two tables: a table about the Houses, and a table to store data about the staff with their house allocation. The table descriptions are as follows:

- HOUSE(houseName, imageURL)
- STAFF(staffID, name, department, houseName*)

The primary key is indicated by underlining one or more attributes.

The attribute indicated with * denotes a foreign key.

Task 4.1

Write a Python program that uses SQL code to create the database HOUSEALLOCATION.db with the two tables given. Define the primary and foreign keys for each table.

[5]

Task 4.2

The text files STAFF.txt and HOUSE.txt store the comma-separated values for the STAFF and HOUSE tables in the database.

Write a Python program to read in the data from each file and then store each item of data in the correct place in the database.

[2]

[Turn over]

Task 4.3

Write a Python program that prompts the user to enter a department name.

The program should then retrieve and display the following details of all staff members belonging to that department:

- Staff ID
- Staff Name
- House Name

If a staff member's house name is "Unassigned", display as `Unassigned`.

The results should be displayed in three neatly aligned columns with appropriate column headings. [3]

Test your program by running the application with the department `MATH/COMP`. [1]

Save your Jupyter Notebook for Task 4.

Task 4.4

Write a Python program and the necessary files to create a web application that:

- Implements a home page that offers the following menu options:
Display House Logos
Allocate House
Display Teachers by Houses
- Implements the **Display House Logos** page that displays all the Houses and its Logo image on a web page in a table. The table has column headers House and Logo. The images should be displayed with width = "100" and height = "70". The six image files are given in the `static` subfolder.
- Implements the **Allocate House** page with a form that accepts two inputs (name and department that exists) in separate textboxes as well as a Submit button.

If the staff belongs to a House, display:

Teacher `<name>` from `<department>` is already in `<xxx>` house.

Otherwise, randomly assign the staff to one of the houses, update the `STAFF` table and display:

Teacher `<name>` from `<department>` is randomly assigned to `<yyy>` house.

- Implements the **Display Teachers by Houses** page, which displays the complete House Allocation in table format. Each column header shows one of the six House names, and the teachers allocated to that House are listed in the row underneath it. Teachers with no assigned house will not have their names appear in the table.

The resulting web pages for **Display House Logos**, **Allocate House** and **Display Teachers by House** should be accessed from the menu on the home page. [14]

Save your Python program as: `TASK4_4_<your name>_<your class>.py`

Task 4.5

Run the web application to:

- display all the house logos
- allocate the following staff to random houses:
 - o Sandra Chelvan from BIO/PHY
- display the teachers using the Display Teachers by Houses page.

[1]

Save the three webpages output in an output subfolder as:

TASK4_5_logos_<your name>_<your class>.html

TASK4_5_allocate_<your name>_<your class>.html

TASK4_5_teachers_<your name>_<your class>.html

END OF PAPER

BLANK PAGE

BLANK PAGE

BLANK PAGE